Kotlin Backend Developer.Professional

otus.ru

REC Проверить, идет ли запись

Меня хорошо видно && слышно?

Интероперабельность с другими языками

Правила вебинара

Активно
участвуем

Off-topic обсуждаем в
OTUS-Kotlin

Задаем вопрос
в чат или голосом

Вопросы вижу в чате,
могу ответить не сразу

Программа курса

Модуль 1: Введение в Kotlin	Введение	Базовые элементы	Функц. элементы	ООП		
Модуль 2: Расширенные возможности Kotlin	DSL	Корутины	Sequence, Flow	Multiplatform	Interop	Gradle
Модуль 3: Подготовка к разработке	Маркетинг аналитика	Security	Архитектура	DevOps		
Модуль 4: Разработка API	OpenAPI	Модели и маперы	Testing	Monitoring		
Модуль 5: Фреймворки и асинхронный транспорт	Spring	Ktor	WebSocket	RabbitMQ	Kafka	Serverless
Модуль 6: Бизнес-логика проекта	Основы БЛ	Модуль БЛ	Валидация			
Модуль 7: Работа с базами данных	Репози-торий	Конкурен-ция	Postgres	Cassandra	Gremlin	
Модуль 8: Развитие бизнес-логики проекта	CollabAPI	Машина состояний	Auth	Основы ML	ML в бэкенде	
Модуль 9: Проектная работа	Обзор экосист.	Консуль-тация	Защита			

План вебинара

- Kotlin Native
- Java FFI
- Рефлексия

Цели вебинара

После занятия вы:

1. Выполнять интеграцию Kotlin-приложения с библиотеками на других языках
2. Создавать на Kotlin библиотеки для переиспользования в других языках
3. Использовать в JVM-приложениях Native-библиотек

Смысл

Зачем вам это уметь

1. Работать с разными экосистемами
2. Есть много полезных библиотек и фреймворков на других языках
3. Можно использовать преимущества других экосистем

Общая информация

О вебинаре

1. Во всех языках есть инструменты интероперабельности
   a. Разные экосистемы языков
   b. Экосистемы специализированы
2. Может сильно понадобиться в карьере
3. Для полного понимания темы может потребоваться знание соотв. языков
4. Факультативное занятие
   a. ДЗ нет
   b. В учебном коде не используется
5. Даже если не знаете, получите общее представление и возможность изучить потом

Kotlin Native

Особенности

- Позволяет скомпилировать код под конкретную платформу в виде исполняемого файла, static-библиотеки или иного артефакта
  (Apple framework для IOS)

- Функциональная совместимость с C, Objective C (C++ не поддерживается!)

- Целевые платформы:
  - macOS
  - iOS, tvOS, watchOS
  - Linux
  - Прочие

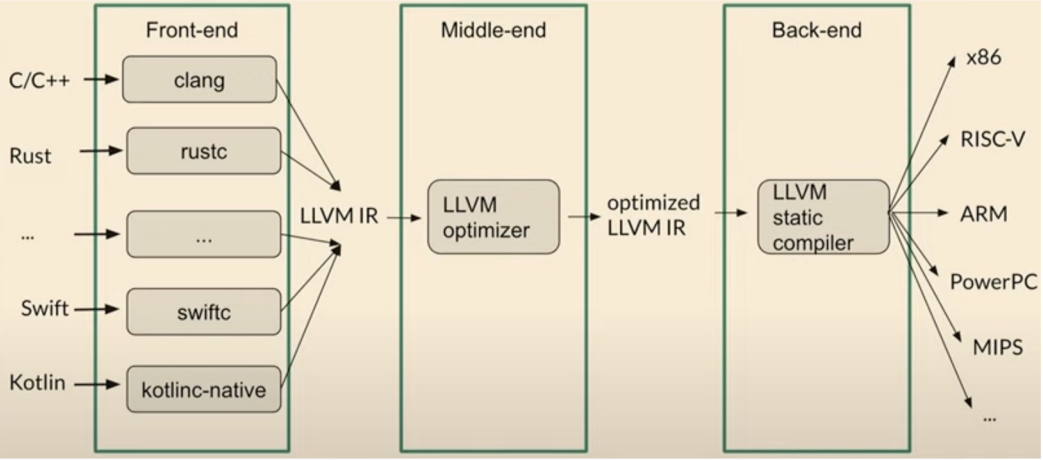
- Поддерживает memory-management для целевых платформ (реализован внутри runtime-части Kotlin/Native)

Компиляция

- Для компиляции используется LLVM https://llvm.org/
- Вначале генерируется IR (Intermediate Representation)
- Из IR производятся целевые артефакты Kotlin/JS и Kotlin/Native

```
Kotlin.String

----jvm-----> java.lang.String
-----js-----> string
---native---> KString
```



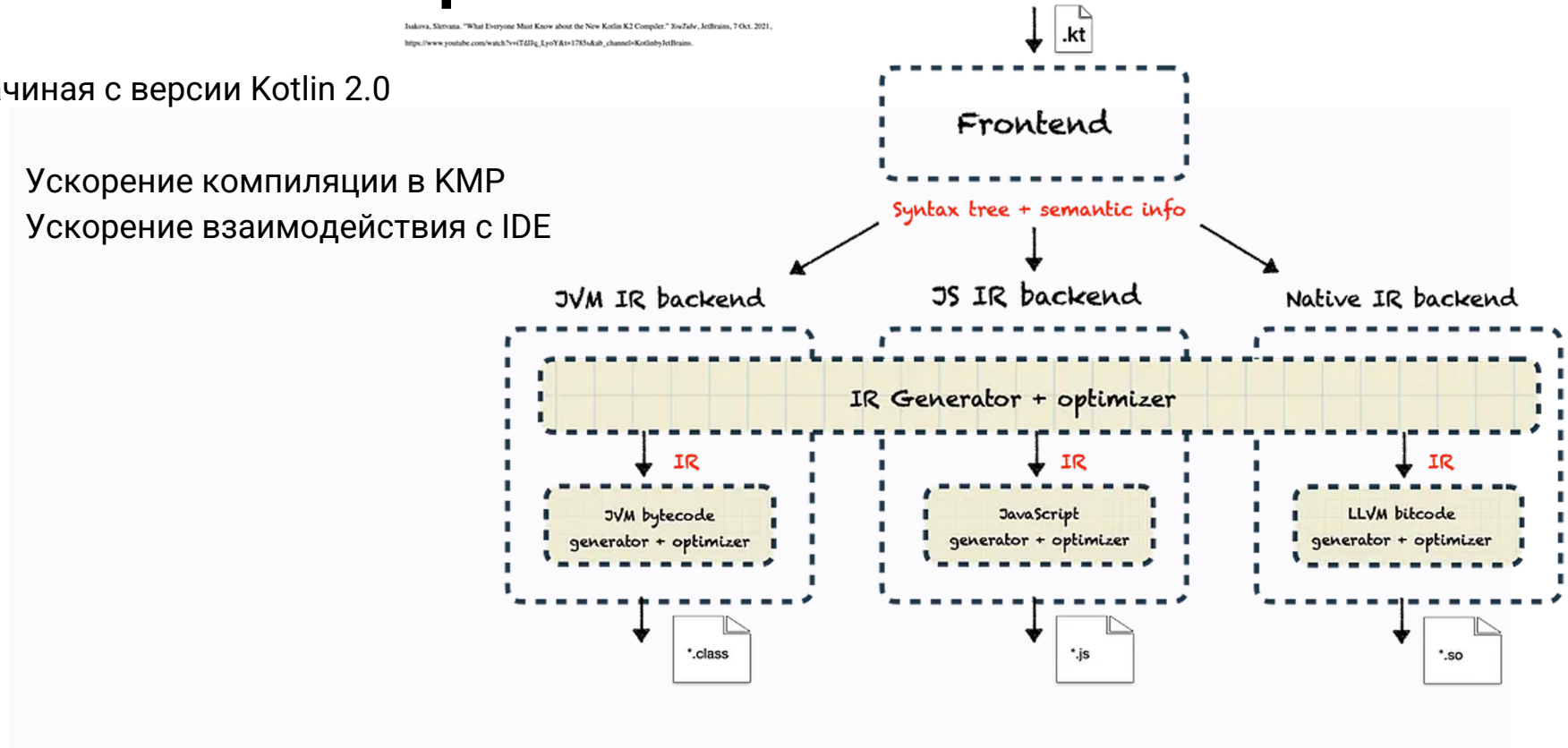
Компоненты компилятора

K2 компилятор

Isakova, Svetlana. "What Everyone Must Know about the New Kotlin K2 Compiler." *YouTube*, JetBrains, 7 Oct. 2021, https://www.youtube.com/watch?v=iTdJfq_LyoY&t=1785s&ab_channel=KotlinbyJetBrains.

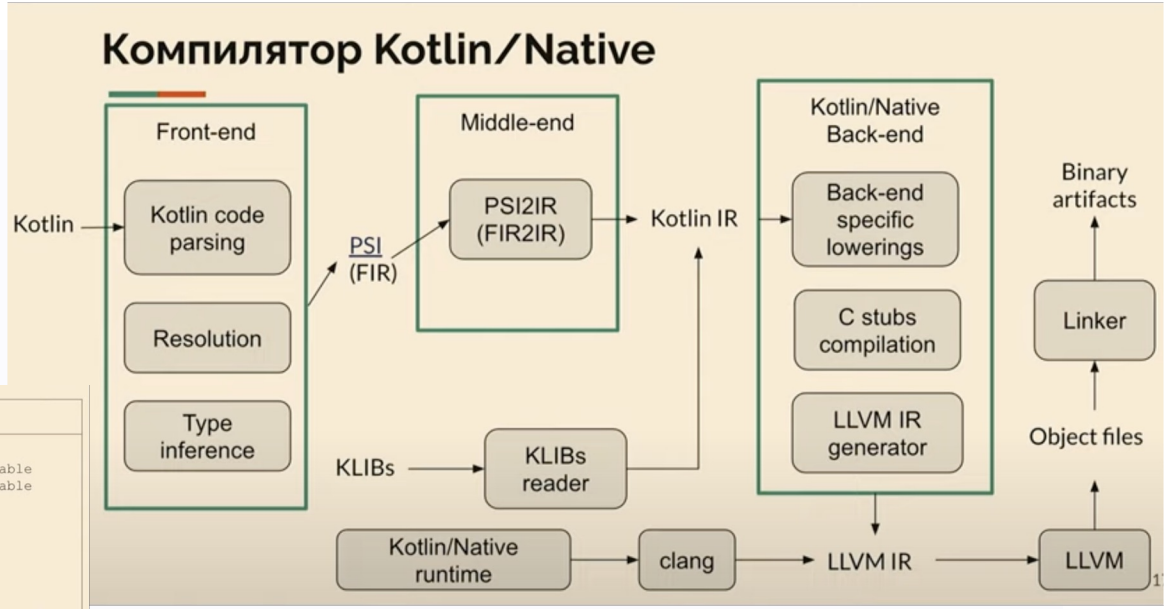
Начиная с версии Kotlin 2.0

- Ускорение компиляции в KMP
- Ускорение взаимодействия с IDE



Детали на Youtube

Доклад от JB
https://www.youtube.com/watch?v=D4ViSA9KVEw
(продублирован в материалах)



Kotlin IR	LLVM IR (LLVM bitcode)
VAR name:sum type:kotlin.Long [var] CONST Long type=kotlin.Long value=0 BLOCK type=kotlin.Unit origin=FOR_LOOP VAR FOR_LOOP_ITERATOR name:tmp0_iterator type:kotlin.collections.IntIterator [val] CALL 'public open fun iterator (): kotlin.collections.IntIterator ... WHILE label=null origin=FOR_LOOP_INNER_WHILE condition: CALL 'public abstract fun hasNext (): kotlin.Boolean ... $this: GET_VAR 'val tmp0_iterator: kotlin.collections.IntIterator [val] ... body: BLOCK type=kotlin.Unit origin=FOR_LOOP_INNER_WHILE ... RETURN type=kotlin.Nothing from='public final fun intArrayIndicesLoop (): kotlin.Long GET_VAR 'var sum: kotlin.Long [var]	do_while_loop: %4 = load i32, i32* %inductionVariable %5 = load i32, i32* %inductionVariable ... call_success: %8 = load %struct.ObjHeader*, %struct.ObjHeader** @"kvar:sample.intArray#internal" %9 = invoke i32 @Kotlin_IntArray_get(%struct.ObjHeader* %8, i32 %4) to label %call_success1 unwind label %cleanup_landingpad call_success1: %10 = sext i32 %9 to i64 %11 = add i64 %7, %10 br label %returnable_block_exit

Ограничения Native

- VM более простая чем Hotspot JVM
- Отсутствие полных возможностей рефлекции
- Отсутствие возможности оптимизировать код во время исполнения
- Использование библиотек – только native библиотеки, либо klib
- Прямой интероперабельности между платформами Kotlin (JVM↔JS↔Native) НЕТ.
  - JVM-Native: JNI – Java Native Interface
  - JVM-JS: Nashorn (Oracle), Rhino (Mozilla Foundation)
  - JS-Native: JSNI
  - etc.

Пример – использование libcurl из C в Kotlin

- Задействуем библиотеку `libcurl` из Kotlin
- Потребуется конфигурационный файл libcurl.def с указанием header-файлов, из которых потребуется сгенерировать Kotlin-заглушки для нативного кода (с помощью инструмента `cinterop`)
- Настройка в Gradle: выявление целевой платформы и применение interop DSL для конфигурации функциональной совместимости Kotlin - C

```
linuxX64("native")
```

```
# list of header files to generate Kotlin stubs
headers = curl/curl.h

compilerOpts.linux_x64 = -I/usr/include -I/usr/include/x86_64-linux-gnu
linkerOpts.osx = -L/opt/local/lib -L/usr/local/opt/curl/lib -lcurl
linkerOpts.linux_x64 = -L/usr/lib/x86_64-linux-gnu -lcurl
```

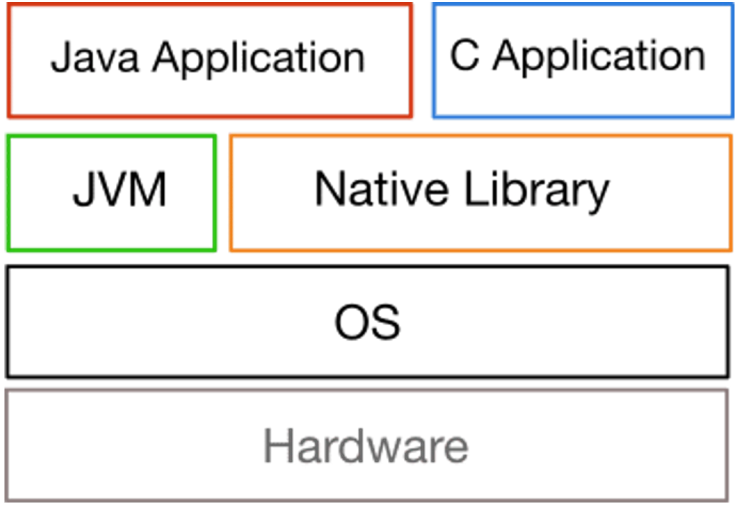
LIVE

Native interop

Java FFI

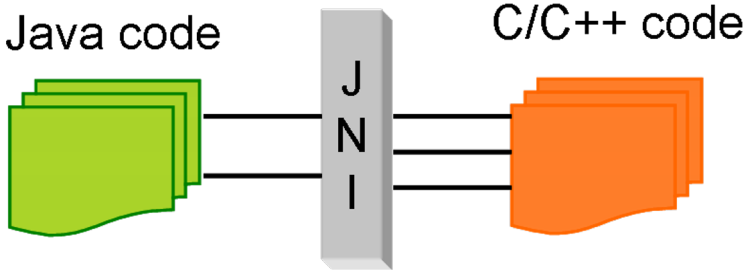
FFI - Foreign Function Interface

1. JNI - Java Native Interface
2. JNA - Java Native Access - поверх JNI
3. JNR - Java Native Runtime - поверх JNI
4. Project Panama - deprecated JNI - новое поколение



JNI - Java Native Interface

1. Обеспечивает вызов нативных библиотек из JVM (Java, Kotlin, Scala)
2. Широко распространен, например врапперы:
   a. OpenCV - обработка видео
   b. Tensorflow - машинное обучение



JNI

1. Пишем класс импорта

```
class RustExample {
    external fun rust_add(a: Int, b: Int): Int

    init {
        System.loadLibrary("rs_jni")
    }
}
```

JNI

```
1. Пишем класс импорта
2. Генерируем файл заголовков (*.h)
```

javah - генерация заголовков из *.java

Используем javap - декомпайлер

С помощью Gradle-task

JNI

1. Пишем класс импорта
2. Генерируем файл заголовков (kotlin.h)
3. Реализуем код для kotlin.h

```
#include "ru_otus_otuskotlin_interop_NativeExample.h"

JNIEXPORT jint JNICALL Java_ru_otus_otuskotlin_interop_NativeExample_rust_1add
  (JNIEnv *jni_env, jobject jo, jint a, jint b) {
  return a + b;
}
```

JNI

1. Пишем класс импорта
2. Генерируем файл заголовков (kotlin.h)
3. Реализуем код для kotlin.h
4. Компилируем библиотеку
5. Все

LIVE

Java Native Interface

Рефлексия и подведение итогов

Цели вебинара

Проверка достижения

1. Выполнять интеграцию Kotlin-приложения с библиотеками на других языках
2. Создавать на Kotlin библиотеки для переиспользования в других языках
3. Использовать в JVM-приложениях Native-библиотек

Тезисы

1.	Kotlin/JS имеет browser и nodejs варианты. Требуется обеспечить интеграционные объявления с помощью extern
2.	Kotlin/Native работает через LLVM, компиляция происходит в IR, а затем в платформу. Для использования требуется импортировать *.h-файлы с помощью cinterop
3.	Большинство языков интегрируются через Native
4.	Интероперабельность JVM-Native обеспечивается через JNI и Panama

Слайд с домашним заданием

1. Создать три композитных проекта: (1) учебные модули; (2) плагин; (3) проектные модули
2. Создать плагины Gradle для сборки подпроектов (1) на базе jvm; (2) на базе multiplatform
3. Создать файл libs.versions.toml и зацепить за него версии зависимостей в плагинах

Сроки выполнения: **до конца модуля**

Список материалов для изучения

1. Документация по Kotlin Multiplatform: https://kotlinlang.org/docs/multiplatform.html
2. https://github.com/AAkira/Kotlin-Multiplatform-Libraries – список библиотек с поддержкой Kotlin Multiplatform
3. Use Kotlin code from JavaScript
4. https://kotlinlang.org/docs/Build a full-stack web app with Kotlin Multiplatform .html
5. Елена Лепилкина - Kotlin/Native: между LLVM и VM (доклад)
6. JNI: https://jonnyzzz.com/blog/2019/12/15/jni-kotlin/
7. Panama: https://denismakogon.github.io/openjdk/panama/2022/05/31/introduction-to-project-panama-part-1.html
8. Kotlin FFI(экспериментальный): https://github.com/whyoleg/ffi-kotlin

Следующий вебинар

Модуль 2: Расширенные возможности Kotlin

Расширенные возможности Gradle

- Ссылка на вебинар будет в ЛК за 15 минут
- Материалы к занятию в ЛК — можно изучать
- Обязательный материал обозначен красной лентой

Заполните, пожалуйста, опрос о занятии по ссылке в чате

Спасибо за внимание! Приходите на следующие вебинары